

Informe Tarea 2 Inteligencia Artificial

Javiera Baeza

Resumen—En el presente informe se describe el desarrollo de una versión autónoma del videojuego Pac-Man, utilizando árboles de comportamiento (Behavior Trees) y máquinas de estados finitos (FSM). Se describen las técnicas implementadas para controlar el comportamiento tanto del Pac-Man como de los fantasmas.

I. INTRODUCCIÓN

Este proyecto explora el uso de FSM y árboles de comportamiento para controlar un agente tipo Pac-Man y sus enemigos (fantasmas), con el objetivo de simular partidas realistas sin intervención de un jugador humano.

Definición máquinas de estado finito Una máquina de estados finitos (FSM) es un modelo de comportamiento que representa un sistema como un conjunto finito de estados posibles, con transiciones definidas entre ellos en función de eventos o condiciones. En el desarrollo de videojuegos, se puede usar para implementar inteligencia artificial simple y estructurada, permitiendo que los agentes (como enemigos o personajes no jugables) cambien su comportamiento de manera predecible.

Definición behavior tree Los árboles de comportamiento (Behavior Trees, BT) son una estructura jerárquica utilizada para modelar comportamientos complejos de agentes en videojuegos de forma modular, flexible y reutilizable. Los BT organizan las acciones y decisiones del agente en forma de nodos que se ejecutan en orden según su tipo (secuencias, selectores, decoradores, entre otros). Esto permite construir comportamientos más expresivos y adaptativos, manteniendo al mismo tiempo una lógica clara y fácil de mantener. Gracias a su capacidad de anidar decisiones y acciones, los árboles de comportamiento son especialmente útiles en entornos dinámicos donde los agentes deben reaccionar a múltiples estímulos del entorno de manera coherente y priorizada.

II. METODOLOGÍA

II-A. Máquina de Estados Finitos

En este proyecto se implementaron máquinas de estados finitos (FSM) tanto para el comportamiento de los fantasmas Blinky y Clyde.

La máquina de estados de Blinky coordina sus transiciones temporales estándar entre los modos de dispersión (Scatter) y persecución (Chase). La modificación incorporada añade el estado de frenesí (Enraged) de proximidad, cuando la distancia euclidiana respecto a Ms. Pacman es inferior a 5 nodos; regresando al modo de persecución normal solo cuando el jugador logra restablecer una distancia segura.

FSM de Blinky

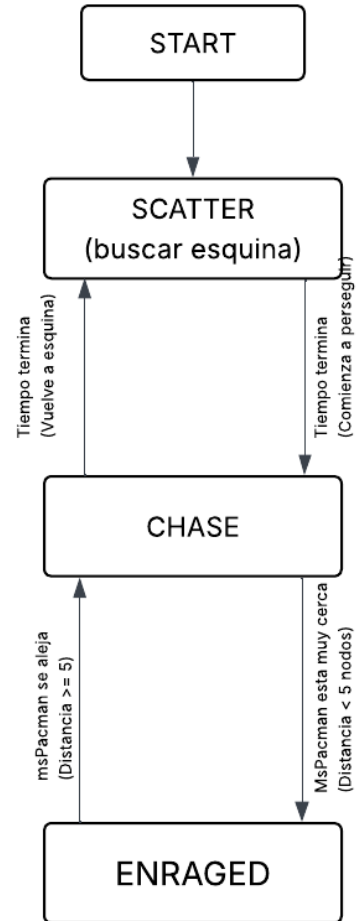


Figura 1: Máquina de Estados Finita (FSM) para el fantasma Blinky

El comportamiento base de Sue/Clyde consiste en vigilar su esquina y salir a perseguir al jugador, volviendo a su zona automáticamente si Ms. Pac-Man se le acerca a menos de 150 unidades. Como modificación propia, le agregamos un estado de emboscada (Ambush). Con esto, en el momento en que el jugador baja de la mitad del mapa (pasando la coordenada Y ¿200), Sue reacciona de inmediato: deja su rol defensivo y se anticipa a los movimientos del jugador, bloqueando los

pasillos del sur para encerrarlo junto a los demás fantasmas.

su comportamiento normal.

FSM de Sue / Clyde

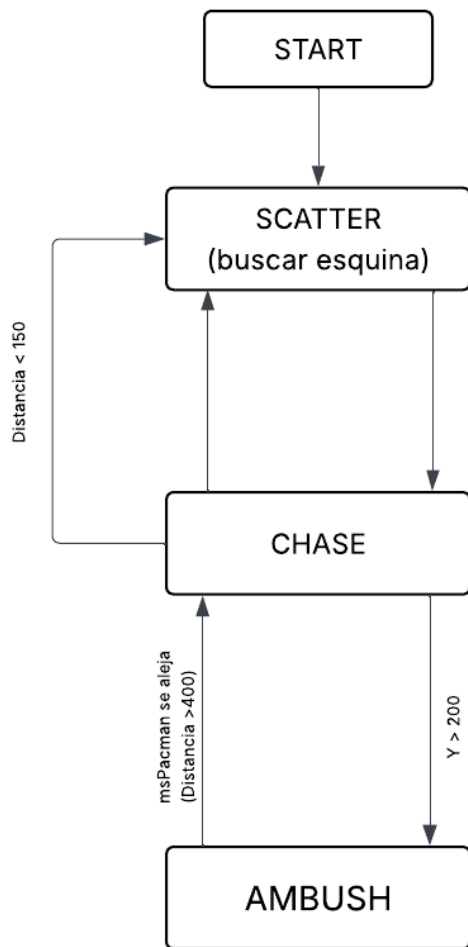


Figura 2: Máquina de Estados Finita (FSM) para el fantasma Clyde

II-B. Árbol de Comportamiento

Los comportamientos de Pinky, Inky y MS. Pacman se diseñaron usando árbol de comportamiento, con nodos de selector y secuencia que permiten elegir entre los estados Chase, Scatter y Frigthed según sea necesario.

Para controlar a Pinky usamos un nodo Selector que va revisando prioridades de izquierda a derecha. Primero evalúa si Ms. Pac-Man se comió una power pill; si es así, el primer filtro activa el estado asustado (Frightened). Si no hay peligro, pasa al segundo filtro para chequear nuestra modificación propia: si Ms. Pac-Man está atrapada en un pasillo sin salida (Pacman Atrapado?), Pinky cambia a un ataque directo para encerrarla. Si ninguna de estas situaciones ocurre, se ejecuta

BT Pinky

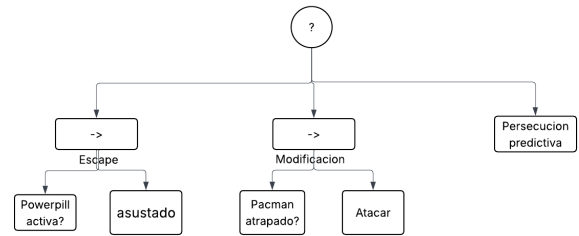


Figura 3: Behavior tree para el fantasma Pinky

El árbol de Inky funciona con un Selector que decide qué hacer según la situación del mapa. La prioridad más alta es la seguridad: si hay una píldora de poder activa, el primer filtro hace que huya inmediatamente (Movimiento Asustado). Si no hay peligro, pasa a evaluar la mejora táctica que le programamos (Blinky Lejos?). Si el código detecta que Blinky se quedó muy atrás y se rompió la estrategia de atacarte en pareja, Inky deja de depender de su compañero y activa la acción propia de Corte de Esquina para intentar emboscar a Ms. Pacman él solo. Si ninguno de estos dos casos ocurre y Blinky está cerca, vuelve a su comportamiento normal de persecución coordinada para acorralar al jugador en equipo.

BT Inky

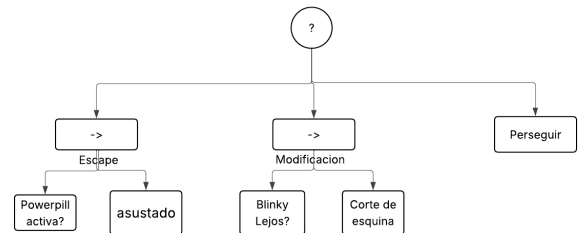


Figura 4: Behavior tree para el fantasma Inky

Para armar el árbol de Ms. Pacman cambiamos la estrategia y priorizamos que no la maten antes de que salga a buscar puntos. El Selector funciona en orden: primero chequea si hay una amenaza (Fantasma Cerca?) para salir corriendo en dirección contraria. Si está a salvo, pasa a revisar si los fantasmas se volvieron azules (Fantasma Comestible?) para aprovechar de comérselos. Si no hay peligro ni fantasmas azules, se va directo a su tarea básica de limpiar el mapa (Recolectar Pastillas). Ahí fue donde metimos mano con nuestro código para solucionar el tema de las esquinas: ahora escanea dinámicamente la comida más próxima y bloquea los giros hacia atrás, evitando que el personaje se quede bugeado o tiritando contra los muros.

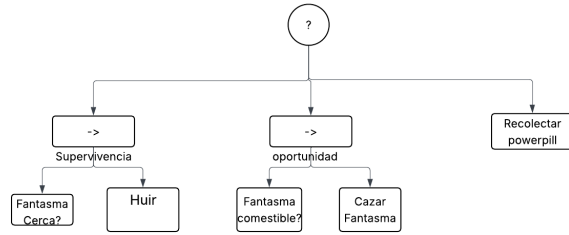


Figura 5: Behavior tree para Ms.Pacman

III. DISCUSIÓN

Los árboles de comportamiento (Behavior Trees) funcionaron muy bien para manejar decisiones complejas y ordenadas por prioridades, lo que se nota claramente en el árbol de Ms. Pac-Man al poner su supervivencia por encima de la recolección, o en la flexibilidad de Inky para evaluar el entorno. Por otro lado, las máquinas de estados finitos (FSM) resultaron ideales para los comportamientos más directos y rutinarios, como las transiciones de tiempo entre Scatter y Chase que usan Blinky y Sue. En nuestro proyecto, aplicar una combinación híbrida entre ambas técnicas nos permitió aprovechar lo mejor de cada una: usamos las FSM para estructurar los estados básicos y predecibles del juego, y los BT para gestionar la toma de decisiones más dinámicas y adaptativas de los agentes, logrando un sistema de inteligencia artificial mucho más robusto y realista para el simulador.

IV. CONCLUSIONES

El desarrollo de este laboratorio permitió contrastar las ventajas prácticas de las Máquinas de Estados Finitos (FSM) y los Árboles de Comportamiento (BT) en un entorno de tiempo real. Las FSM resultaron ideales para los fantasmas de lógica más lineal como Blinky y Sue; la inclusión de estados personalizados (Enraged y Ambush) demostró que es posible aumentar la dificultad del juego de forma reactiva sin elevar el costo computacional. Por otro lado, los BT ofrecieron la flexibilidad jerárquica necesaria para coordinar decisiones complejas y planes de respaldo, lo que permitió que Inky tomara la iniciativa de forma solitaria al perder a su compañero.

El código fuente y la implementación completa de este proyecto se encuentran disponibles en el repositorio público de GitHub: https://github.com/Berirluwu/Tarea-2_IA.git.